

Traffic Simulation System Documentation

Architecture, Concurrency, and Analytics Report

Quy Trang Duong (1646906)
Quang Minh Khoa Le (1645066)
Nguyen Vo Minh Khoi (1647325)
Duc Khang Nguyen (1646894)
Thanh Hung Phan (1650754)

January 18, 2026

Contents

1	Introduction	4
1.1	Project Overview	4
1.2	Motivation and Problem Statement	4
1.3	Project Objectives and Goals	4
2	System Architecture & Concurrency	4
2.1	High-Level MVC Architecture	4
2.2	Concurrency: The Producer-Consumer Pattern	5
2.2.1	The Producer (Simulation Thread)	5
2.2.2	The Data Packet (Immutable State)	5
2.2.3	The "Fan-Out" Distribution Strategy	5
2.2.4	The Consumers	6
3	Infrastructure & Map Management	6
3.1	Architectural Hierarchy & Relationships	6
3.2	Class Implementation Details	6
3.2.1	MapManager: The Orchestrator	6
3.2.2	EdgeClass: The Road Segment	7
3.2.3	LaneClass: The Atomic Unit	7
3.2.4	JunctionClass: The Intersection	7
3.3	Data Instantiation Flow	7
4	Vehicle & Agent Hierarchy	8
4.1	Vehicle Hierarchy	8
4.2	Pedestrians	8
4.3	VehicleClass Specification	8
4.4	VehicleManager Specification	9
4.5	StatisticsManager Specification	10
4.6	ChartWindow Specification	12

5	Traffic Light Control System	13
5.1	Traffic Light Architecture	14
5.2	TrafficLightManager Responsibilities	14
5.3	TrafficLightManager Initialisation	15
5.4	Switching to the Next Traffic Light Phase	15
5.5	Setting Traffic Light Color and Duration	15
5.5.1	Setting Phase Duration	15
5.5.2	Setting Traffic Light Color	16
6	Data Analytics Reporting	16
6.1	Statistical Analysis	16
6.2	Reporting & Charts	16
6.2.1	Export Vehicle CSV	17
6.2.2	Export Edge CSV	17
6.2.3	Export PDF Report	17
6.2.4	PDF Threading Model and JavaFX Synchronisation	18
7	Visualization & User Interface	18
7.1	Architectural Layering	18
7.2	Renderer & Performance Caching Strategy	18
7.3	Polymorphic Rendering Implementation	19
7.4	Infrastructure Visualization	19
7.4.1	Lanes and Junctions	19
7.4.2	Traffic Lights	19
7.5	User Interaction UX	20
7.5.1	Traffic Light Control	20
7.5.2	Vehicle Inspection	20
7.6	Coordinate Transformation Utilities	20
7.6.1	The Coordinate Mismatch	20
7.6.2	Transformation Logic	20
7.7	Color Space Conversion	21
7.7.1	JavaFX to Simulation (Encoding)	21
7.7.2	Simulation to JavaFX (Decoding & Bitwise Masking)	21
7.8	Visual Filtering	21
8	Map Interaction & Navigation Logic	22
8.1	Event Handling Architecture	22
8.2	Multimodal Input Controls	22
8.3	Pivot-Based Transformation Logic	22
9	Dynamic Visualization Filtering	23
9.1	Functional Architecture	23
9.2	Predicate Construction Strategy	23
9.3	Cross-Layer Integration	24
9.3.1	View Layer (Renderer)	24
9.3.2	Model Layer (Stream API)	25
9.4	Color Space Interoperability	25

10 Implementation Challenges & Design Trade-offs	25
10.1 Technical Challenges & Optimization	25
10.1.1 Rendering Optimization: The State-Diff Caching Strategy	25
10.1.2 Coordinate System Transformation	26
10.2 Concurrency Architecture & Race Condition Prevention	26
10.2.1 The Producer-Consumer Pattern	26
10.3 Architectural Trade-offs	27
10.3.1 The "Fan-Out" Strategy: Responsiveness vs. Synchronization . .	27
10.3.2 JavaFX Scene Graph vs. Canvas API	27
11 Error Handling Robustness	27
11.1 Custom Exception: SumoException	27
11.1.1 The "Fatal" Flag Design Pattern	28
11.2 Integration in Control Logic	29
12 Core Features Requirement Compliance	29
12.1 Core System Features	29
12.1.1 1. High-Performance Visualization	29
12.1.2 2. Real-Time Data Analysis	29
12.1.3 3. Infrastructure Control	29
12.2 Rubric Fulfillment Matrix	30
12.3 Intelligent use of AI in Asset Creation	30
13 Conclusion	30
14 Future Improvements	31
14.1 Adaptive Traffic Control (Machine Learning)	31
14.2 Heatmap Visualization	31
15 Team Contributions & Work Distribution	31

1 Introduction

1.1 Project Overview

This document details the implementation of a JavaFX-based Traffic Simulation integrated with SUMO. The system visualizes real-time traffic data, calculates analytics, and generates reports.

1.2 Motivation and Problem Statement

Traffic congestion and air pollution are one of the big problems in metropolises. This could be resolved through lots of optimization, testing, and real-life experiments. However, these experiments are costly and take time to show results. This is where our Application can come into assist solving the problem. With our contribution, we believe that the optimization process can be push faster and more efficient.

1.3 Project Objectives and Goals

- Utilizing SUMO for smooth Simulation
- Implemented in a clean code manner and Object-Oriented approach
- Interactive UI
- Optimization for smooth Simulation
- Apply the concept of Multi-threading
- Support sufficient testing components
- Support Statistical Analysis and Exports

2 System Architecture & Concurrency

2.1 High-Level MVC Architecture

The application strictly follows the Model-View-Controller (MVC) design pattern to ensure separation of concerns:

- **Model:** The `SimulationManager` (along with `VehicleClass` and `MapManager`) handles the business logic, state management, and interaction with the SUMO backend via TraCI.
- **View:** The `Renderer` class and JavaFX FXML files ('Main.fxml') are responsible strictly for visual presentation. They do not calculate simulation logic.
- **Controller:** The `MainController` acts as the central coordinator. It initializes the threads, handles user inputs (clicks, buttons), and mediates the data flow between the Model and the View.

2.2 Concurrency: The Producer-Consumer Pattern

To ensure the application remains responsive while processing heavy simulation logic, the system architecture decouples the computational backend from the graphical frontend. We implemented a multi-threaded Producer-Consumer pattern, preventing the heavy SUMO calculations from blocking the JavaFX Application Thread.

2.2.1 The Producer (Simulation Thread)

The `MainController` initializes a dedicated background thread (managed via an `ExecutorService`) that acts as the **Producer**. This thread runs the core simulation loop independently of the UI's frame rate. In each iteration, it performs three atomic steps:

1. **Advance:** It commands the SUMO backend to advance the simulation by one time step (`simManager.step()`).
2. **Capture:** It retrieves the raw data (vehicle positions, traffic light states) from the TraCI interface.
3. **Encapsulate:** It deep-copies this data into an immutable `SimulationState` object, effectively creating a "snapshot" of the world at that specific tick.

2.2.2 The Data Packet (Immutable State)

Thread safety is strictly enforced by the `SimulationState` class. In Java, objects are passed by reference; passing raw model objects (which are being modified by the loop) to the UI thread would inevitably lead to `ConcurrentModificationException` or visual artifacts (tearing).

To solve this, `SimulationState` serves as an **Immutable Snapshot**. Once created, its internal maps (Vehicles, Traffic Lights) cannot be modified. This allows multiple threads to read the same state object simultaneously without the need for complex synchronization locks.

2.2.3 The "Fan-Out" Distribution Strategy

A key architectural decision was the implementation of a "Fan-Out" strategy within the `MainController`. Rather than having a single consumer, the Producer distributes the `SimulationState` into two independent, thread-safe `BlockingQueue` channels.

```
1 // Inside the Simulation Thread Loop:
2 simManager.step();
3 SimulationState state = simManager.getState();
4
5 // FAN-OUT STRATEGY:
6 // The same immutable state is offered to two independent pipelines
7 this.uiQueue.offerState(state); // Pipeline A: For the Renderer
8 this.statQueue.offerState(state); // Pipeline B: For Statistics/Logging
```

Listing 1: The Fan-Out Logic in `MainController.java`

This "Double Buffering" approach provides significant reliability benefits:

1. **Decoupling of Concerns:** The `Renderer` (Consumer A) and the `StatisticsManager` (Consumer B) operate at different speed. If the statistical calculations take 200ms

but rendering takes only 16ms, the UI remains smooth because it consumes from a separate queue.

2. **Fault Tolerance:** A lag spike in the visualization layer (e.g., due to the user zooming in heavily) does not block the data logging thread, ensuring that the exported reports remain accurate and complete.

2.2.4 The Consumers

The system employs two distinct consumers that process the queues populated by the Fan-Out strategy:

- **Consumer 1: The Visualizer (UI Thread)**
Implemented via a `JavaFX AnimationTimer`. In every frame pulse (60Hz), it polls the `uiQueue`. If a new state is available, it triggers the `Renderer` to update the scene graph. This polling is non-blocking to ensure the interface never freezes.
- **Consumer 2: The Stats Engine (Background Thread)**
A separate worker thread that blocks on the `statQueue`. It takes every available state to update real-time charts and accumulate historical data for the PDF report.

3 Infrastructure & Map Management

The infrastructure layer serves as the static backbone of the simulation. It allows the application to abstract away the raw string-based commands of the TraCI interface into a structured, Object-Oriented hierarchy. This layer is managed entirely by the `model.infrastructure` package.

3.1 Architectural Hierarchy & Relationships

The data model follows a strict hierarchy designed to mirror real-world road network topology. We utilized two primary design relationships:

- **Composition (Edge $\rightarrow$ Lane):** The relationship between `EdgeClass` and `LaneClass` is a strict **Composition**. In the code, the `EdgeClass` constructor explicitly calculates the number of lanes and instantiates the `LaneClass` objects immediately. A Lane cannot exist in the model without its parent Edge, and the Edge is responsible for the lifecycle of its Lanes.
- **Aggregation (MapManager $\rightarrow$ Elements):** The `MapManager` acts as a container (Aggregate) for the entire network. It holds references to all Edges, Lanes, and Junctions in searchable `HashMap` collections, allowing for $O(1)$ lookup complexity by ID.

3.2 Class Implementation Details

3.2.1 MapManager: The Orchestrator

The `MapManager` acts as the Data Access Object (DAO) for the static map environment. Upon initialization, it queries SUMO for the global lists of Edge IDs and Junction IDs.

- **Boundary Calculation:** It retrieves the map's bounding box (`SumoBoundingBox`), which is crucial for the `CoordinateConverter` to scale the visual output to the screen size.
- **Instantiation Loop:** It iterates through the ID lists, creating high-level Java objects (`'EdgeClass'`, `'JunctionClass'`) and populating the internal maps.

3.2.2 EdgeClass: The Road Segment

Represents a directed connection between two nodes. Beyond storing geometry, the `EdgeClass` performs **Logic Propagation**.

- When initialized, it iterates through its child `Lane` objects.
- If *any* child lane allows a specific vehicle type (e.g., "bicycle"), the Edge marks itself as allowing that type (`'this.allowsBicycle = true'`).
- This optimizations prevents the route finder from needing to scan every single lane during pathfinding checks.

3.2.3 LaneClass: The Atomic Unit

This class encapsulates the lowest level of infrastructure data. It parses the raw permission strings from SUMO into strongly-typed Java primitives.

- **Permission Parsing:** The raw data from SUMO returns a list of allowed strings (e.g., `["passenger", "taxi"]`). The `LaneClass` processes this into boolean flags (`allowsPassenger`, `allowsBicycle`) for faster boolean checks during the simulation loop.
- **Geometry:** It holds the `SumoGeometry` (a list of 2D points) required for rendering the lane's curve on the canvas.

3.2.4 JunctionClass: The Intersection

A lightweight wrapper that represents the nodes in the graph. It stores the intersection's ID and its polygon shape (`SumoGeometry`), which represents the "walking area" or the physical intersection space in the visualization.

3.3 Data Instantiation Flow

To ensure data integrity, the infrastructure is loaded synchronously at startup:

1. **Connect:** `MapManager` establishes the TraCI connection.
2. **Fetch IDs:** It retrieves the complete list of Edge IDs and Junction IDs from the SUMO kernel.
3. **Recursive Build:**
 - The Manager creates an `EdgeClass`.
 - The `EdgeClass` constructor immediately queries SUMO for its lane count.

- The `EdgeClass` loops and creates `LaneClass` objects.
4. **Indexing:** Finally, all objects are placed into `Map<String, T>` collections to facilitate instant retrieval by the `SimulationManager` and `Renderer`.

4 Vehicle & Agent Hierarchy

The system uses the `MeansOfTransportation` interface to standardize moving objects. `MeansOfTransportation` is an abstract class for all moving objects in the vehicle simulation (for `VehicleClass` and `PedestrianClass`).

4.1 Vehicle Hierarchy

- `VehicleClass` (Abstract Base)
- `CarClass`, `BusClass`, `BikeClass` (Concrete Implementations)

4.2 Pedestrians

`PedestrianClass` implements logic specific to sidewalks and crossings.

4.3 VehicleClass Specification

Overview and Objective

The class `model.vehicles.VehicleClass` serves as an abstract base class for `BikeClass`, `BusClass`, and `CarClass`. Each object of `VehicleClass` represents a snapshot of a single vehicle containing attributes such as `id`, `color`, `position`, `speed`, `edgeId`, `angle`, and `departure`.

Attributes

The specific attributes of a vehicle snapshot are defined as follows:

`id` (`String`)

The unique identifier of the vehicle taken from the SUMO simulation.

Format: Text (e.g., “veh0”)

`color` (`SumoColor`)

The color of the vehicle representation inside the SUMO simulation.

Format: Byte `rgba`

`position` (`SumoPosition2D`)

The 2D coordinates of the vehicle inside the SUMO simulation.

Unit: (x, y) in meters

`speed` (`double`)

The current speed of the vehicle inside the SUMO simulation.

Unit: m/s

`edgeId` (String)

The identifier of the edge (road segment) the vehicle is currently travelling on.

Format: Text (e.g., “0”)

`angle` (double)

The travelling direction of the vehicle inside the SUMO simulation.

Unit: Degree (0–360)

`departure` (double)

The time that the vehicle starts travelling in the SUMO simulation.

Format: Simulation Timestep

Design Principles

- **Immutability:** Once an object serves with its attributes defined, the attributes are not changeable.
- **Access:** Data of attributes are only available through getter functions.

4.4 VehicleManager Specification

Overview and Objective

The class `model.vehicles.VehicleManager` serves as a manager responsible for managing the snapshot of all vehicles at each time step in the simulation. Its main tasks are:

- **Data synchronization:** Periodic retrieval of global vehicle data in the SUMO simulation.
- **Data storage:** Storing the up-to-date state of all vehicles in the SUMO simulation.
- **Simulation Interaction:** Adding vehicles into the SUMO simulation.

Update Cycle (step method)

The update process consists of the following steps:

1. Request the list of IDs of all vehicles within the SUMO simulation.
2. Clear the cache of old data.
3. Loop through all IDs; for each ID, send a request for detailed data such as speed, position, color, edge, angle, and departure time.
4. Create a new `VehicleClass` instance with the retrieved data and save this instance in the hashmap.

Error Handling

- If the connection to the SUMO simulation is stopped or closed, the `VehicleManager` stops updating data and clears the list of vehicles to avoid inconsistency.
- Errors resulting from requests for vehicle attributes are captured and processed without bringing the whole simulation to an abrupt stop or crash.

Important Methods

`step()`

Triggers the synchronization process. This should be called in every simulation step.

`getVehiclesData()`

Returns a copy of the data of all vehicles inside the simulation (Map: `ID` → `VehicleClass`). Returning a copy prevents external modifications to the data inside the `VehicleManager`.

`injectVehicle(...)`

Adds a new vehicle based on the route ID and type ID into the SUMO simulation and sets the color for the vehicle.

`injectPerson(...)`

Adds a new person based on the edge list, type ID, person ID, and color into the SUMO simulation.

4.5 StatisticsManager Specification

Overview and Objective

The class `StatisticsManager` is responsible for calculating real-time statistics inside the vehicle simulation. It aggregates raw data from the vehicles to derive traffic metrics (e.g., average vehicle speed, vehicle density per edge ID). The class updates at every time step of the simulation to provide up-to-date metrics regarding the SUMO simulation.

Its responsibilities include:

- Storage of a snapshot of all active vehicles inside the simulation.
- Calculation of the average speed of all vehicles.
- Identifying congested edges.
- Calculating vehicle densities per edge.
- Producing a histogram of the distribution of travel times.

Attributes (Internal State)

The internal state maintains the data required for calculations:

`vehiclesData` (`Map<String, VehicleClass>`)

Stores the actual vehicle data, referenced by the vehicle ID.

`step` (`int`)

The actual time step of the simulation.

Construction and Initialization

`StatisticsManager()`

Initializes a new instance of the manager.

Action: Creates a new empty `HashMap` for `vehiclesData`.

Method Specification

Data Update

`step(Map<String, VehicleClass> vehiclesInfo, int step)`

Updates the internal state of the manager with the latest data from the simulation loop.

- **vehiclesInfo**: Map of all active vehicles.
- **step**: The current simulation step.

Analytical Methods

`avgVehiclesSpeed()`

Calculates the average speed of all active vehicles (in m/s).

Logic:

- Sum `vehicles.getSpeed()` for all vehicles.
- Divide by the number of vehicles.

Edge case: Returns 0.0 and logs an error using `System.err` when there are no active vehicles.

`findCongestionSpots()`

Finds the edges which are congested.

Definition: An edge is identified as congested if the average speed of all vehicles within this edge is under $5.0 m/s$.

Algorithm:

1. Group all vehicles based on the `edgeId`.
2. Calculate the average speed per edge.
3. Filter out edges where the average speed is < 5.0 .

`calculateVehicleDensity()`

Calculates the number of vehicles per edge.

Return: `Map<String, Integer>` (Key: Edge-ID, Value: number of vehicles).

`calculateTravelTimeDistribution(int binSizeSeconds)`

Creates a histogram of the actual travel time of all active vehicles.

Return: `Map<String, Integer>` (TreeMap).

- **Key:** Time interval as a string (e.g., "10-20").
- **Value:** Number of vehicles in this interval.
- **Sorting:** The map is a `TreeMap` which sorts intervals based on the lower bound.

Calculation of travel time:

$$\text{Time} = (\text{step} \times 0.1) - \text{vehicle.getDeparture}()$$

(Note: The simulation step corresponds to 0.1 seconds).

Dependencies

- `model.vehicles.VehicleClass`: Required to access `getSpeed()`, `getEdgeID()`, and `getDeparture()`.
- `java.util.*`: Use of `Map`, `List`, `HashMap`, `TreeMap`, `ArrayList`.

4.6 ChartWindow Specification

Overview and Objective

The class `ChartWindow` is a graphical user interface based on JavaFX, designed for the visualization of live statistics of the simulation. It creates a separate window containing three different diagrams to graphically demonstrate the development of the simulation (speed, density, travel time).

Its responsibilities include:

- Initialization and configuration of the primary window (`Stage`) and the scene (`Scene`).
- Creation and layout of three diagrams:
 - Line chart for the average vehicle speed within the network.
 - Bar chart for the vehicle density per edge.
 - Bar chart for the distribution of travel time.
- Thread-safe realization of the diagram updates using data from the simulation thread.
- Management of the window's visibility.

Attributes (GUI Components)

The class maintains references to the JavaFX components required for visualization:

`stage` (`Stage`)

The main window of the application.

`speedChart` (`LineChart<Number, Number>`)

The line diagram for visualizing the average speed.

`densityChart` (`BarChart<String, Number>`)

The bar chart for visualizing vehicle density.

`densitySeries` (`XYChart.Series<String, Number>`)

Data series for the density chart.

Axes: X: Edge-ID, Y: Number of vehicles.

`speedSeries` (`XYChart.Series<Number, Number>`)

Data series for the speed chart.

Axes: X: Time step, Y: Speed.

`travelTimeChart` (`BarChart<String, Number>`)

The bar chart for visualizing the travel time distribution.

`travelTimeSeries` (`XYChart.Series<String, Number>`)

Data series for the travel time chart.

Axes: X: Time interval, Y: Frequency.

Construction and Initialization

`ChartWindow()`

Creates a new instance of the statistic window.

Action:

- Initializes the `Stage` and sets the window title.
- Calls `initCharts()` for diagram configuration.
- Creates the layout (`VBox`) and the scene.

`initCharts()`

Configures the axes (labels) and instantiates the diagrams and the `XYChart.Series` objects.

`show()`

Implements simple logic for managing the window:

- If the window is not open, it is shown.
- If it is already open, it is brought to the front using `toFront()`.

Method Specification

`updateData(...)`

This is the main method for the data connection between the simulation and the view.

Parameters:

- `int currentStep`: The current time step.
- `double avgSpeed`: The Y-value for the line chart (average speed).
- `Map<String, Integer> densityMap`: Data source for the density bar chart.
- `Map<String, Integer> travelTimeMap`: Data source for the travel time distribution bar chart.

5 Traffic Light Control System

The traffic light control system manages, visualises, and controls all traffic light junctions within the SUMO simulation. It acts as the connection between the simulation backend (SUMO via TraCI) and the graphical user interface, enabling both real-time rendering and user interaction.

The system is implemented using two main components:

- **TrafficLightManager**: manages all traffic light junctions globally.
- **TrafficLightClass**: represents a single traffic light connection within a junction.

Since one traffic light junction can control multiple lane connections, each junction corresponds to multiple traffic light objects in the system.

5.1 Traffic Light Architecture

In SUMO, a traffic light junction controls several incoming and outgoing lane connections. Each connection has its own signal state (red, yellow, or green) and is internally represented by a `SumoLink` object.

To reflect this structure, the system uses the following hierarchy:

- One **traffic light junction** (TLS ID)
 - Multiple **TrafficlightClass** objects

Each `TrafficlightClass` stores:

- The associated `SumoLink`
- The host junction ID
- The link index within the traffic light definition
- Source, destination, and intermediate lane IDs
- Spatial position of the traffic light (`SumoPosition2D`)

This abstraction allows each visible traffic light to be handled individually, while still remaining part of its hosting junction.

5.2 TrafficlightManager Responsibilities

The `TrafficlightManager` acts as the central controller of the traffic light system. Its responsibilities include:

- Retrieving all traffic light junction IDs from SUMO
- Extracting controlled links for each junction
- Creating and storing `TrafficlightClass` objects
- Providing traffic light data for rendering
- Querying current signal states and phase information
- Switching traffic light phases
- Modifying signal colors and phase durations

All communication with the simulation is performed through a shared TraCI connection.

5.3 TrafficlightManager Initialisation

During initialisation, the `TrafficlightManager` performs the following steps:

1. Retrieve all traffic light junction IDs using:

```
Trafficlight.getIDList()
```

2. For each junction, retrieve its controlled links using:

```
Trafficlight.getControlledLinks(tlsId)
```

3. For every controlled link:

- Create a `TrafficlightClass` object
- Store the host junction ID
- Assign the corresponding link index

4. Determine the spatial position of the traffic light by:

- Retrieving the originating lane geometry
- Extracting the final coordinate of the lane shape
- Storing it as a `SumoPosition2D`

5.4 Switching to the Next Traffic Light Phase

The system allows manual switching to the next traffic light phase. This is achieved using SUMO's built-in command:

```
Trafficlight.setPhaseDuration(tlsId, 0.0)
```

Setting the phase duration to zero immediately ends the current phase and forces the junction to advance to the next phase defined in the active program.

5.5 Setting Traffic Light Color and Duration

Users can modify traffic lights in two ways (both options can be applied independently or together):

- Adjust the phase duration
- Force a specific signal color

5.5.1 Setting Phase Duration

Setting Phase Duration modifies the remaining time of the current phase for the entire traffic light junction. This phase duration is updated using the TraCI command:

```
Trafficlight.setPhaseDuration(tlsId, duration)
```

5.5.2 Setting Traffic Light Color

Changing the signal color requires adjusting the traffic light program. The procedure is as follows:

1. Retrieve the complete traffic light definition:

```
Trafficlight.getCompleteRedYellowGreenDefinition()
```

2. Identify the phase in which the selected `TrafficlightClass` displays the desired signal color.
3. Construct a new traffic light program where:
 - The selected phase becomes the first phase
 - All following phases are appended in order
 - Remaining earlier phases are added at the end

4. Overwrite the existing program using:

```
Trafficlight.setCompleteRedYellowGreenDefinition()
```

5. Activate the new program starting from phase index zero.

This method keeps the original phase cycle intact while ensuring that the active phase matches the user-selected signal color.

6 Data Analytics Reporting

6.1 Statistical Analysis

The `StatManager` performs real-time calculations on the `SimulationState`, including:

- Average Network Speed
- Total CO2 Emissions
- Vehicle Density

6.2 Reporting & Charts

The reporting subsystem provides functionality for exporting simulation data and generating structured analysis reports. It enables both raw data extraction in CSV format and summarized visual reporting in PDF format.

The export functionality is implemented through the `ReportManager` class.

6.2.1 Export Vehicle CSV

The vehicle CSV export contains detailed information for each simulated vehicle. The exported attributes include:

- `vehicle_id`
- `speed`
- `time_from_spawn`
- `color`
- `type`

If vehicle filtering is enabled by the user interface, only the selected subset of vehicles is included in the exported file. Otherwise, all active vehicles in the simulation are exported.

6.2.2 Export Edge CSV

The edge CSV export provides aggregated traffic information for road segments within the network.

Each exported entry contains:

- `edge_id`
- `width`
- `density`
- `average_speed`

An optional congestion filter can be enabled, allowing the user to restrict the exported edges based on the following criteria:

- Vehicle density $\geq$ `min_vehicle_density`
- Average speed $\leq$ `max_average_speed`

Both threshold values are provided dynamically through user interface controls.

6.2.3 Export PDF Report

The PDF export generates a summarized simulation report containing both numerical statistics and graphical visualisations.

The report includes:

- Average network speed
- Total number of active vehicles
- Current simulation timestep
- Vehicle density distribution chart
- Travel time distribution chart

6.2.4 PDF Threading Model and JavaFX Synchronisation

PDF generation is executed in a dedicated background thread to avoid blocking the simulation loop and user interface.

However, chart rendering relies on JavaFX components, which must be executed exclusively on the JavaFX Application Thread.

For this reason, the following execution sequence is used:

1. User initiates export via button click (JavaFX UI thread)
2. A dedicated PDF export thread is started
3. The PDF thread performs all computationally intensive operations
4. When a chart image is required:
 - The PDF thread requests rendering via `Platform.runLater`
 - The JavaFX thread constructs the chart and generates a snapshot image
5. The PDF thread waits safely for completion using thread synchronization
6. PDF generation resumes once the image is available

7 Visualization & User Interface

The Visualization layer acts as the passive consumer in the MVC architecture. It is responsible for translating the abstract mathematical state of the simulation (managed by the `SimulationManager`) into a graphical representation on the JavaFX Application Thread. The core of this functionality lies within the `view.Renderer` class.

7.1 Architectural Layering

To ensure correct visual occlusion (e.g., vehicles appear *above* roads but *below* traffic lights), the rendering logic is divided into distinct JavaFX `Pane` layers:

1. **Infrastructure Layer (Bottom):** Renders static elements such as `Lane` and `Junction` geometries.
2. **Vehicle Layer (Middle):** Renders dynamic agents (Cars, Buses, Pedestrians).
3. **Traffic Light Layer (Top):** Renders interactive traffic signaling systems.

7.2 Renderer & Performance Caching Strategy

Rendering a traffic simulation requires a high frame rate (typically 30-60 FPS). A naive implementation that instantiates new JavaFX `Node` objects for every vehicle in every frame would cause severe performance degradation due to heap allocation overhead and frequent Java Garbage Collection (GC) pauses.

To mitigate this, the `Renderer` implements a **State-Diff Caching Strategy**:

- **The Cache:** A `HashMap<String, Node>` is maintained, mapping a unique entity ID (e.g., "veh.42") to its active JavaFX `Node`.

- **Update Logic:** In each render cycle, the algorithm iterates through the current `SimulationState`:
 - **Cache Hit:** If the ID exists in the map, the existing `Node` is retrieved. Only its mutable properties (`Translation X/Y`, `Rotation`) are updated to match the new simulation state. This operation is $O(1)$ and generates zero garbage.
 - **Cache Miss:** If the ID is new, the renderer invokes the polymorphic `getShape()` method (see Sec. 7.3) to create the node, adds it to the Scene Graph, and stores it in the cache.
- **Stale Object Cleanup:** A secondary pass identifies IDs present in the cache but missing from the current simulation state (vehicles that have exited the map). These nodes are removed from the JavaFX scene and the cache to prevent memory leaks.

7.3 Polymorphic Rendering Implementation

The `Renderer` adheres to the Open/Closed Principle by remaining agnostic to the specific type of vehicle being drawn. It relies on the `MeansOfTransportation` interface.

- **Abstraction:** The interface defines the method `Node getShape(Color c)`.
- **Implementation:**
 - `CarClass`: Returns a `Rectangle` representing a standard chassis.
 - `PedestrianClass`: Returns a `Group` consisting of an `Ellipse` (shoulders) and a `Circle` (head).

When `drawVehicles()` is called, the renderer treats all objects uniformly as `MeansOfTransportation`, delegating the geometric definition to the specific object implementation.

7.4 Infrastructure Visualization

7.4.1 Lanes and Junctions

Road infrastructure is rendered by extracting `SumoGeometry` points from the TraCI API. These geo-coordinates are transformed into screen pixels (via `CoordinateConverter`) and rendered as JavaFX `Polygon` or `Polyline` shapes. Since infrastructure is static, this drawing process occurs only once during initialization to save processing power.

7.4.2 Traffic Lights

Traffic lights are visualized as composite `Group` nodes containing three circles (Red, Yellow, Green).

- **State Visualization:** The renderer reads the character code from the SUMO Link State (e.g., 'r', 'G', 'y') and dynamically adjusts the fill color of the corresponding circle to appear "lit" or "dim."

7.5 User Interaction UX

The application supports rich user interaction through event handlers attached to the rendered nodes.

7.5.1 Traffic Light Control

To enable manual intervention, traffic lights are interactive:

- **Hover Effect:** When the mouse hovers over a traffic light, a `DropShadow` effect is applied, and the cursor changes to a hand icon. This provides visual feedback that the element is clickable.
- **Click Event:** Clicking a traffic light triggers a callback to the `MainController`, which populates the control sidebar with that specific junction’s details, allowing the user to modify phase durations.

7.5.2 Vehicle Inspection

Users can click on any moving vehicle. The renderer attaches a `setOnMouseClicked` handler to every vehicle node. When triggered, the specific `Vehicle` object is passed to the controller, logging its live statistics (Speed, Edge ID, Destination) to the console or info panel.

7.6 Coordinate Transformation Utilities

The simulation engine (SUMO) and the visualization engine (JavaFX) utilize fundamentally different coordinate systems. The `CoordinateConverter` class bridges this gap by performing linear affine transformations to map simulation physics to screen pixels.

7.6.1 The Coordinate Mismatch

- **SUMO (Cartesian):** Uses a standard mathematical grid where the origin (0,0) is typically at the **Bottom-Left**. The Y-axis increases *upwards*.
- **JavaFX (Screen):** Uses a computer graphics grid where the origin (0,0) is at the **Top-Left**. The Y-axis increases *downwards*.

7.6.2 Transformation Logic

To render the map correctly, the converter first calculates the map boundaries (`mapMinX`, `mapMaxY`) via the `setBound()` method. It then applies a scaling factor (`scale`) representing pixels per meter.

The transformation formulas implemented in `toScreenX` and `toScreenY` are:

$$X_{screen} = (X_{sumo} - X_{min_offset}) \times S \quad (1)$$

$$Y_{screen} = (Y_{max_offset} - Y_{sumo}) \times S \quad (2)$$

Notably, the Y-axis calculation involves an inversion: (`mapMaxY - sumoY`). This subtraction flips the axis, ensuring that "North" in the simulation correctly points to the top of the JavaFX window, rather than the bottom.

7.7 Color Space Conversion

The `ColorConverter` utility handles data type discrepancies between the simulation core and the rendering engine. While JavaFX manages colors as normalized doubles (0.0–1.0), the backend simulation (TraCI/SUMO) utilizes standard 8-bit integers (0–255) packed into objects.

7.7.1 JavaFX to Simulation (Encoding)

When a user selects a vehicle color in the UI, the `toSumoColor(Color fxColor)` method converts the normalized JavaFX values into the integer range required by SUMO:

$$Val_{sumo} = \lfloor Val_{fx} \times 255 \rfloor \quad (3)$$

This allows the abstract `SumoColor` object to store Red, Green, Blue, and Alpha values as integers from 0 to 255.

7.7.2 Simulation to JavaFX (Decoding & Bitwise Masking)

When rendering vehicles, the system must convert `SumoColor` back to JavaFX `Color`. A critical technical challenge here is Java’s handling of the `byte` primitive.

In Java, the `byte` type is **signed**, representing values from -128 to 127. However, RGB color channels are standard **unsigned** 8-bit values (0 to 255). Directly casting a color value like 200 (light gray) to a Java byte would result in a negative number (e.g., -56), causing rendering errors.

To solve this, the `toFXColor` method employs a bitwise AND mask with `0xFF`:

```
int r = sumoColor.r & 0xFF;
int g = sumoColor.g & 0xFF;
int b = sumoColor.b & 0xFF;
```

Mechanism:

1. The bitwise operator `&` promotes the signed `byte` to a 32-bit `int`.
2. The mask `0xFF` (binary `0000...11111111`) zeros out the higher 24 bits.
3. This effectively interprets the binary data as an unsigned integer (0–255), which is then normalized back to the 0.0–1.0 range required by JavaFX (e.g., `a / 255.0` for opacity).

7.8 Visual Filtering

The renderer supports dynamic filtering using Java’s `Predicate` interface. The `drawVehicles` method accepts a `Predicate<MeansOfTransportation>` `filter`. Before rendering or updating any vehicle, the system evaluates this predicate.

- If `filter.test(vehicle)` returns **false**, the vehicle’s node is made invisible or removed from the scene.
- This allows features such as "Show only Electric Vehicles" or "Hide Pedestrians" to be implemented efficiently without altering the underlying simulation data.

8 Map Interaction & Navigation Logic

To provide a fluid User Experience (UX) akin to modern GIS applications, the simulation implements a comprehensive interaction layer via the `MapInteractionHandler` class. This component supports multimodal input, seamlessly bridging traditional mouse controls with modern multi-touch trackpad gestures to manipulate the view (Pan, Zoom, Rotate).

8.1 Event Handling Architecture

The interaction logic employs a decoupled architecture to prevent coordinate jitter and ensure smooth transformations:

- **Input Node (Listener):** The static UI wrapper (e.g., `StackPane`) that captures raw input events (Mouse, Scroll, Zoom, Rotate) in screen coordinates.
- **Target Node (Transformable):** The `Group` containing the map elements (Lanes, Vehicles), to which the affine transformations (Scale, Translate, Rotate) are applied.

8.2 Multimodal Input Controls

The handler registers specific listeners to support both mouse users and laptop trackpad users:

- **Panning (Translation):**
 - *Mouse:* Primary Button (Left-Click) + Drag.
 - *Logic:* Calculates the delta between the current mouse position and the anchor point established on `MousePressed` to update `TranslateX/Y`.
- **Zooming (Scaling):**
 - *Mouse:* Scroll Wheel (handled via `ScrollEvent`).
 - *Trackpad:* Pinch-to-Zoom gesture (handled via `ZoomEvent`).
 - *Constraints:* Zoom level is clamped between `MIN_SCALE` (0.1x) and `MAX_SCALE` (15x).
- **Rotation:**
 - *Mouse:* Secondary Button (Right-Click) + Drag. Uses a sensitivity factor (`MOUSE_ROTATION_SENSITIVITY = 0.8`) to map pixel drag distance to degrees.
 - *Trackpad:* Two-finger rotate gesture (handled via `RotateEvent`).

8.3 Pivot-Based Transformation Logic

A critical UX requirement is "Pivot-Aware" transformation—zooming or rotating specifically around the cursor or finger position rather than the top-left corner (0,0). The `MapInteractionHandler` implements a drift-correction algorithm to achieve this.

The mathematical procedure for both `zoomToPivot` and `rotateAroundPivot` is as follows:

1. **Pivot Capture:** The cursor’s screen coordinate (P_{screen}) is converted to the map’s local coordinate space before transformation:

$$P_{map} = \text{targetNode.sceneToLocal}(P_{screen}) \quad (4)$$

2. **Apply Transform:** The new properties (Scale or Rotation angle) are applied to the target node.
3. **Calculate Drift:** Modifying the geometry shifts the visual location of P_{map} . We calculate its new position on the screen:

$$P_{new_screen} = \text{targetNode.localToScene}(P_{map}) \quad (5)$$

4. **Corrective Translation:** The difference (drift) between the new screen position and the original input position is subtracted from the translation to "snap" the map back under the cursor:

$$\Delta_{drift} = P_{new_screen} - P_{screen} \quad (6)$$

$$T_{final} = T_{current} - \Delta_{drift} \quad (7)$$

This ensures that if a user points at a specific intersection and zooms in, that intersection remains exactly under their mouse cursor throughout the animation.

9 Dynamic Visualization Filtering

To allow users to analyze specific subsets of traffic (e.g., "Show only Red cars moving faster than 10 m/s"), the application implements a dynamic filtering system. Rather than scattering conditional 'if/else' logic throughout the rendering loop, the system leverages Java’s Functional Programming capabilities, specifically the `java.util.function.Predicate` interface, to encapsulate complex filtering rules into a single portable object.

9.1 Functional Architecture

The core of the filtering logic is the `Predicate<MeansOfTransportation>`. This functional interface represents a boolean-valued function of one argument. It allows the `MainController` to define *what* should be visible, while the `Renderer` and `SimulationManager` only need to know *how* to execute the test.

9.2 Predicate Construction Strategy

In the `MainController`, the user’s UI selections are compiled into a single, comprehensive lambda expression. Unlike simple predicate chaining, the implementation uses a block-body lambda to handle state dependencies (such as whether a filter is active or inactive) and necessary type conversions (SUMO colors to JavaFX colors).

The logic follows a "Short-Circuit" pattern: if a specific filter category (e.g., Speed) is disabled in the UI, the logic automatically passes `true` for that category.

```

1 Predicate<MeansOfTransportation> filterRule = v -> {
2     // 1. Safety Check
3     if (v == null) return false;
4
5     // 2. Speed Evaluation
6     // Logic: If the speed filter is NOT active, the vehicle passes
7     // automatically.
8     // Otherwise, its speed must be less than the threshold.
9     boolean speedPass = !filterSpeedActive || v.getSpeed() <=
10    maxSpeedCriteria;
11
12    // 3. Color Evaluation
13    boolean colorPass = true;
14    if (filterColorActive) {
15        if (selectedFXColors.isEmpty()) {
16            // Active filter but no colors selected = Hide All
17            colorPass = false;
18        } else {
19            SumoColor sumoColor = v.getColor();
20            if (sumoColor != null) {
21                // Bridge Pattern: Convert Model Color to View Color
22                // for comparison
23                Color vehicleFXColor = ColorConverter.toFXColor(
24                sumoColor);
25                colorPass = selectedFXColors.contains(vehicleFXColor);
26            } else {
27                colorPass = false;
28            }
29        }
30    }
31
32    // 4. Final Intersection
33    return speedPass && colorPass;
34 };

```

Listing 2: Complex Predicate Construction in MainController

9.3 Cross-Layer Integration

Once this `filterRule` object is created, it is propagated to other layers of the application, ensuring a "Single Source of Truth" for visibility.

9.3.1 View Layer (Renderer)

The `Renderer.renderMeansOfTransportation` method accepts the predicate. It does not need to know about sliders or checkboxes; it simply invokes `filterRule.test(vehicle)`.

- **Efficiency:** The check is performed in $O(1)$ time per vehicle.
- **Behavior:** If `.test()` returns `false`, the renderer performs a "Logical Removal"—the vehicle's JavaFX node is removed from the scene graph to free up rendering resources, effectively hiding it from the user.

9.3.2 Model Layer (Stream API)

The same predicate is passed to the `SimulationManager` to generate statistics. This ensures that if the user filters the map to show only "Red Cars," the generated CSV report or chart also contains data for *only* "Red Cars."

The `getFilteredVehicleIDs` method utilizes Java Streams to filter the data collection declaratively:

```
1 public List<String> getFilteredVehicleIDs(Predicate<
   MeansOfTransportation> criteria, SimulationState state) {
2     if (state == null) return new ArrayList<>();
3
4     return state.getMeansOfTransportations().values().stream()
5         .filter(criteria) // Apply the specific
   rule created in Controller
6         .map(MeansOfTransportation::getId) // Extract IDs of
   passing vehicles
7         .toList();
8 }
```

Listing 3: Stream Filtering in `SimulationManager`

9.4 Color Space Interoperability

A key challenge addressed inside the predicate (Listing 2) is the discrepancy between data models. The `MeansOfTransportation` object holds a `SumoColor` (Integer-based), while the UI selection set contains `javafx.scene.paint.Color` (Double-based).

The predicate acts as the integration point, invoking `ColorConverter.toFXColor()` on the fly. This conversion allows the backend data to remain lightweight (integers) while the frontend filtering logic operates on rich objects, maintaining strict separation of concerns until the exact moment of comparison.

10 Implementation Challenges & Design Trade-offs

This section outlines the significant technical hurdles encountered during the development of the Traffic Simulator and the architectural decisions made to balance performance, thread safety, and code maintainability.

10.1 Technical Challenges & Optimization

10.1.1 Rendering Optimization: The State-Diff Caching Strategy

The Problem (Naive Implementation): In the initial development phase, the rendering logic followed a naive approach: clearing the entire JavaFX Pane (`pane.getChildren().clear()`) and re-instantiating all vehicle shapes (Rectangles, Circles) from scratch for every simulation tick (60Hz).

Impact: This approach created thousands of short-lived objects per second, placing immense pressure on the Java Garbage Collector (GC). The result was significant "GC Pauses," causing the UI to stutter visually despite the backend simulation running smoothly.

The Solution (Object Pooling): We implemented a persistent Caching Strategy

within `Renderer.java`. A `HashMap<String, Node>` acts as a lookup table for active visual elements. The rendering loop was refactored to perform a "State-Diff" update:

- **Check:** For every vehicle ID in the current frame, the system checks the cache.
- **Update (Hit):** If the node exists, we modify its mutable properties (Translation X/Y, Rotation) to match the new state. This avoids memory allocation entirely for existing vehicles.
- **Create (Miss):** A new Node is instantiated only when a vehicle physically enters the map.
- **Prune:** A secondary pass removes nodes only when their corresponding ID is absent from the simulation state (vehicle left the map).

This optimization stabilized the frame rate and reduced CPU usage by avoiding redundant object creation.

10.1.2 Coordinate System Transformation

Problem: The simulation backend (SUMO) utilizes a Cartesian coordinate system where the origin (0,0) is at the bottom-left, and the Y-axis increases upwards. Conversely, the JavaFX UI coordinate system places the origin at the top-left, with the Y-axis increasing downwards. Furthermore, different SUMO maps have vastly different spatial dimensions.

Solution: We implemented a `CoordinateConverter` utility class. It utilizes an affine transformation to map simulation meters to screen pixels dynamically. It calculates a scaling factor based on the `MapManager`'s bounding box and the current window size, ensuring the network fits the viewport regardless of aspect ratio or resolution.

10.2 Concurrency Architecture & Race Condition Prevention

10.2.1 The Producer-Consumer Pattern

Challenge: The most critical architectural challenge was handling data sharing between the **Simulation Thread** (backend) and the **JavaFX Application Thread** (frontend).

Risk: If the `Renderer` attempted to read the vehicle list while the `Simulation Manager` was simultaneously writing to it, a `ConcurrentModificationException` (Race Condition) would occur, crashing the application.

Solution: We implemented a strict **Producer-Consumer Pattern** using a thread-safe `ArrayBlockingQueue`.

1. **The Producer (`SimulationManager`):** In every tick, the manager creates an *immutable deep copy* of all vehicle and traffic light data, wraps it in a `SimulationState` object, and pushes it to the `SimulationQueue`.
2. **The Consumer (`Renderer`):** The JavaFX thread polls this queue. It never accesses the live simulation objects directly; it only consumes the immutable snapshots provided by the queue.

This architecture completely decouples the simulation logic from the rendering logic, guaranteeing thread safety without the need for complex `synchronized` blocks that could deadlock the UI.

10.3 Architectural Trade-offs

10.3.1 The "Fan-Out" Strategy: Responsiveness vs. Synchronization

Decision: The `MainController` splits the stream of simulation states into two separate queues: a `uiQueue` for the visualizer and a `statQueue` for the real-time charts.

Trade-off:

- **Benefit (Responsiveness):** This ensures that heavy statistical calculations (e.g., standard deviation of travel times) in the `StatisticsManager` do not block the rendering loop. If the charts lag due to computation load, the 2D map remains smooth and responsive.
- **Drawback (Temporal Drift):** This introduces a slight desynchronization. It is possible for the Visualizer to display Tick 100 while the Chart is still processing Tick 98.
- **Justification:** In a real-time simulation, UI fluidity is paramount. The temporal drift is typically in the order of milliseconds and is imperceptible to the user, making this an acceptable compromise for a better User Experience.

10.3.2 JavaFX Scene Graph vs. Canvas API

Decision: We chose to render vehicles as individual JavaFX Nodes (Scene Graph) rather than drawing pixels directly onto a `Canvas`.

Trade-off:

- **Benefit (Interaction):** The Scene Graph provides built-in event handling. We can attach `setOnMouseClicked` listeners directly to a `Vehicle` object. Implementing this on a `Canvas` would require complex manual "hit-testing" mathematics to map a mouse click coordinate to a moving vehicle.
- **Drawback (Scalability):** The Scene Graph is heavier on memory and slower than the `Canvas` API when rendering extremely high object counts ($> 10,000$).
- **Justification:** Since the project scope involves urban simulation (typically $< 1,000$ active vehicles) and prioritizes user interaction (inspecting vehicle details), the Scene Graph's ease of use outweighed the raw performance benefits of the `Canvas` API.

11 Error Handling Robustness

To ensure the simulation remains stable under unpredictable conditions (e.g., network timeout with SUMO, invalid map data), we implemented a robust error handling strategy centered around a custom domain-specific exception.

11.1 Custom Exception: `SumoException`

Following the project requirements, we implemented `util.SumoException`, a custom class extending the standard Java `Exception` (Checked Exception).

Unlike generic Java exceptions, `SumoException` introduces a **Severity Flag** architecture.

11.1.1 The "Fatal" Flag Design Pattern

The core innovation in this class is the `boolean fatal` field. This allows the system to distinguish between:

1. **Recoverable Errors (`Fatal = false`):** *Example:* A specific vehicle ID cannot be found in the current tick. *Action:* The error is logged to the console, but the simulation loop continues.
2. **Critical Failures (`Fatal = true`):** *Example:* The TraCI connection to the SUMO server is lost. *Action:* The application halts gracefully, alerting the user and closing resources to prevent zombie processes.

```
1 package util;
2
3 /**
4  * A domain-specific exception for SUMO simulation errors.
5  * Includes a 'severity level' to distinguish between crashes and
6  * warnings.
7  */
8 public class SumoException extends Exception {
9
10     private static final long serialVersionUID = 1L;
11     private final boolean fatal;
12
13     /**
14      * Constructor for Minor Errors (Non-Fatal).
15      * Used for logic issues like "Invalid Vehicle ID".
16      */
17     public SumoException(String message) {
18         super(message);
19         this.fatal = false;
20     }
21
22     /**
23      * Constructor for Critical Errors.
24      * Used for system failures like "Connection Lost".
25      * @param isFatal If true, the Simulation Manager will stop the
26      * loop.
27      */
28     public SumoException(String message, boolean isFatal) {
29         super(message);
30         this.fatal = isFatal;
31     }
32
33     // Chained Exception Constructor for preserving stack traces
34     public SumoException(String message, Throwable cause, boolean
35     isFatal) {
36         super(message, cause);
37         this.fatal = isFatal;
38     }
39
40     public boolean isFatal() {
41         return fatal;
42     }
43 }
```

Listing 4: Implementation of the Custom SumoException

11.2 Integration in Control Logic

In the `SimulationManager`, this exception is caught explicitly. The manager checks `e.isFatal()`. If true, it triggers the `stopSimulation()` sequence; otherwise, it simply logs the warning via `Log4j2` and proceeds to the next simulation tick. This prevents the entire GUI from crashing due to a single malformed data packet.

12 Core Features Requirement Compliance

This section summarizes the implemented functionality and explicitly maps the technical architecture to the project requirements defined in the grading rubric.

12.1 Core System Features

12.1.1 1. High-Performance Visualization

The application renders a real-time representation of the SUMO traffic simulation.

- **Vector Graphics:** Utilizes `SVGPath` for vehicle rendering to ensure crisp visuals at any zoom level without pixelation.
- **Smooth Navigation:** Implements "Google Maps-style" interaction (Pivot Zooming, Panning, and Rotation) via the `MapInteractionHandler`.
- **Visual Caching:** Uses a `HashMap` caching strategy in the `Renderer` to maintain 60 FPS performance by minimizing object instantiation overhead.

12.1.2 2. Real-Time Data Analysis

Beyond visual observation, the system provides analytical tools:

- **Live Charts:** The `StatisticsManager` feeds data to JavaFX `LineCharts` and `BarCharts`, visualizing "Active Vehicles over Time" and "Average Network Speed."
- **Reporting:** Includes a `ReportManager` that generates a downloadable PDF summary of the simulation run using `Apache PDFBox`.

12.1.3 3. Infrastructure Control

- **Traffic Light Management:** Users can click on any junction to view its phase program. The `TrafficLightManager` allows modifying phase durations dynamically during runtime.
- **Entity Inspection:** Clicking on a vehicle pauses the inspector to show live telemetry (Speed, ID, Destination Edge).

12.2 Rubric Fulfillment Matrix

The following table outlines how specific project requirements were met through architectural decisions:

Requirement	Implementation Details
Object-Orientation (10 pts)	Implemented the <code>MeansOfTransportation</code> interface to allow polymorphic handling of <code>CarClass</code> , <code>BusClass</code> , and <code>PedestrianClass</code> within the <code>Renderer</code> and <code>VehicleManager</code> . Used Composition for <code>Edge-Lane</code> relationships.
Collections (5 pts)	Utilized <code>HashMap<String, Node></code> for $O(1)$ render caching, <code>ArrayBlockingQueue</code> for thread-safe data transfer, and <code>ArrayList</code> for maintaining ID lists.
Threads (5 pts)	Implemented a strict Producer-Consumer Pattern . Thread 1 (Simulation) produces state; Thread 2 (JavaFX) consumes it; Thread 3 (Stats) processes data for charts (The "Fan-Out" strategy).
Error Handling (5 pts)	Created <code>util.SumoException</code> (Custom Exception) with a <code>fatal</code> flag logic. Used <code>try-catch</code> blocks in <code>TrafficLightManager</code> to log errors via <code>Log4j2</code> without crashing the UI.
Streams & Files (5 pts)	Implemented <code>ReportManager</code> to write binary PDF files using streams. The application also reads configuration properties and CSS files via input streams.
Clean Code (10 pts)	Adhered to naming conventions (CamelCase), removed "Magic Numbers" (e.g., extracted constants in <code>PedestrianClass</code>), and strictly separated logic using the MVC pattern.

Table 1: Project Requirement Compliance Matrix

12.3 Intelligent use of AI in Asset Creation

In addition to the mandatory requirements, we integrated **Generative AI** into the workflow to generate low-poly SVG paths for vehicles and to ensure CSS accessibility compliance. This demonstrates an adaptation of modern development tools.

13 Conclusion

The development of this Traffic Simulation System successfully demonstrates the power of integrating high-level Object-Oriented design with low-level simulation kernels. By wrapping the complex string-based commands of the SUMO TraCI interface into strongly-typed Java objects (`EdgeClass`, `VehicleClass`, `LaneClass`), we achieved a codebase that is both robust and maintainable.

The project highlights several key architectural victories:

- **Concurrency & Performance:** The implementation of the **Producer-Consumer** pattern with the "Fan-Out" strategy proved critical. It allowed the application to maintain a responsive UI (60 FPS) while simultaneously processing heavy statistical

data on background threads, effectively solving the "frozen UI" problem common in simulation software.

- **Data Integrity:** The decision to use an **Immutable SimulationState** snapshot ensured thread safety without the need for complex and error-prone synchronization locks, preventing **ConcurrentModificationException** crashes.
- **Clean Code Principles:** Adhering to the SOLID principles, particularly the Single Responsibility Principle (e.g., separating **Renderer**, **MapManager**, and **StatisticsManager**), has resulted in a modular system where individual components can be upgraded or replaced without breaking the simulation core.

Ultimately, the application serves not only as a visualizer but as a foundation for complex traffic analysis, proving that JavaFX is a viable platform for developing high-performance scientific tools.

14 Future Improvements

While the current system fulfills all functional requirements, the modular architecture opens several avenues for future enhancements:

14.1 Adaptive Traffic Control (Machine Learning)

Currently, the **TrafficLightManager** allows for manual or fixed-time control. A future iteration could implement a Reinforcement Learning (RL) agent. By feeding the **StatisticsManager**'s real-time density data into a neural network, the system could dynamically adjust phase durations to minimize network-wide waiting times.

14.2 Heatmap Visualization

Instead of just showing moving vehicles, the **Renderer** could be upgraded to overlay a "Congestion Heatmap." By accumulating the speed data from **LaneClass** over time, we could color-code road segments (Green → Red) to visually highlight bottlenecks to the user instantly.

15 Team Contributions & Work Distribution

The development of the Traffic Simulation System was distributed among five team members, with each member taking ownership of specific architectural modules to ensure parallel development and specialization.

Team Member	Primary Role	Key Contributions & Modules
Hung	Core Execution, Threading & UI Integration	MainController (Orchestration), SimulationManager (High-level control), Thread Design (ExecutorService), MapInteractionHandler (Input handling), CoordinateConverter (World-to-Screen transformation), Primary FXML Layout.
Trang	Vehicle/Infrastructure Rendering	Renderer module (Visualizing Vehicles, Lanes, Junctions), Secondary FXML GUI elements, ColorConverter , Interactive Event Handling (Implementing Consumers for Object Clicks), Vehicle Injection Logic.
Khoi	Vehicle Modeling & Data Analysis	VehicleManager (Lifecycle management), MeansOfTransportation interface (Polymorphism), VehicleClass hierarchy, StatisticsManager (Data aggregation), SubVehicle logic, Real-time Charts and FXML Statistics interface.
Khang	Traffic Control, Rendering & Reporting	TrafficlightManager (Control logic), TrafficlightClass (Model), Traffic Light Visualization, FXML Control Pane (Phase duration logic), ReportManager (PDF and CSV Generation).
Khoa	Data Pipeline, Map Init & Data Filtering	SimulationQueue (Producer-Consumer buffer), SimulationState (Immutable snapshots), Stress Testing Logic, MapManager (Edge/Lane parsing), Advanced Data Filtering (Implemented via Java Predicate).

Table 2: Work Distribution and Module Ownership